PROJECT REPORT • RTL DESIGN AND VERIFICATION

8-bit RISC Processor Core with a 5-Stage Pipeline

Verilog HDL design, self-checking testbench and Vivado synthesis

IF → ID → EX → MEM → WB | forwarding, load-use stall, branch flush



Author	Tran Duy (SE193143)
Course	Semiconductor & VLSI Industry Readiness Program
Instructor	Mr Shivashish Pal
Design tool	AMD Vivado 2026.1 (XSim simulator, Vivado synthesis)
Target device	Xilinx Artix-7 xc7a35tcpg236-1
Language	Verilog-2001 (synthesizable RTL)
Date	October 2026

Reference material and project specification provided by GUVI × HCLTech

Table of Contents

1. Introduction . . . 4

1.1 Objectives . . . 4

1.2 Scope . . . 4

1.3 Tools . . . 4

2. Background . . . 5

2.1 RISC and the Harvard architecture . . . 5

2.2 Combinational and sequential logic . . . 5

2.3 Pipelining . . . 5

2.4 Hazards . . . 5

3. Specification and Instruction Set . . . 6

3.1 Processor specification . . . 6

3.2 Instruction formats . . . 6

3.3 The 16 instructions . . . 7

4. Architecture . . . 8

4.1 Top-level organisation and file hierarchy . . . 8

4.2 The five-stage datapath . . . 9

4.3 Pipeline registers and valid bits . . . 9

5. RTL Implementation . . . 10

5.1 ALU (alu.v) . . . 10

5.2 Register file (regfile.v) . . . 10

5.3 Instruction and data memories (imem.v, dmem.v) . . . 10

5.4 Control unit (control_unit.v) . . . 11

5.5 Hazard unit (hazard_unit.v) . . . 11

5.6 Top level (risc8_5stage_core.v) . . . 11

6. Hazard Handling in Detail . . . 13

6.1 Forwarding and the register-file bypass . . . 13

6.2 Load-use stall . . . 13

6.3 Branch flush . . . 13

6.4 Halting . . . 13

7. Verification Methodology . . . 14

7.1 Testbench structure . . . 14

7.2 Test program . . . 14

7.3 Vivado project setup . . . 15

8. Simulation Results . . . 16

8.1 Cycle-by-cycle trace . . . 16

8.2 Automatic check result . . . 16
8.3 Complete waveform . . . 17
8.4 Forwarding (55–135 ns) . . . 17
8.5 Branch flush (100–185 ns) . . . 18
8.6 Load-use stall (405–485 ns) . . . 18
8.7 Halting (465–545 ns) . . . 19
8.8 Final register file contents . . . 19

9. Synthesis Results . . . 21
9.1 Elaborated design . . . 21
9.2 Resource utilization . . . 21

10. Performance Analysis . . . 22

11. Challenges and Lessons Learned . . . 23

12. Conclusion . . . 23
References . . . 23

Appendix A. Test Program Listing . . . 24

Appendix B. Repository Structure . . . 24

1. Introduction

This report describes the design, verification and synthesis of an **8-bit RISC processor core with a classic five-stage pipeline** (Instruction Fetch, Instruction Decode, Execute, Memory access and Write-back). The processor is written in synthesizable Verilog-2001, verified with a self-checking testbench in the Vivado XSim simulator, and synthesized for a Xilinx Artix-7 FPGA.

The instruction set is a custom 8-bit RISC ISA (it is not RISC-V, whose smallest base is 32-bit). It has 16 instructions, 8 general-purpose registers and separate instruction and data memories. Because several instructions are in flight at the same time, the core must detect and resolve **pipeline hazards**: it forwards results between stages, stalls for one cycle when a load is followed immediately by a use of the loaded value, and flushes wrongly fetched instructions after a taken branch.

1.1 Objectives

- Understand how an instruction set architecture is turned into a pipelined micro-architecture.
- Write and read RTL Verilog for combinational blocks (ALU, decoder, hazard unit) and sequential blocks (register file, pipeline registers, program counter).
- Write a **self-checking testbench**: clock and reset generation, a per-cycle monitor, automatic comparison against expected values and a PASS/FAIL summary.
- Use waveforms to observe forwarding, load-use stalls, branch flushes and halting cycle by cycle.
- Run Vivado synthesis and interpret the resource utilization of the design.

1.2 Scope

The focus of this project is **Verilog coding and testbench-based verification**. The work covers RTL design, behavioral simulation and logic synthesis. Place-and-route timing closure and programming a physical FPGA board are outside the scope of this report.

1.3 Tools

Item	Used in this project
HDL	Verilog-2001, synthesizable subset for the core; behavioral constructs in the testbench
Design suite	AMD Vivado 2026.1: project management, XSim behavioral simulation, RTL elaboration, synthesis
Target part	Xilinx Artix-7 xc7a35tcpg236-1 (the FPGA used on the Basys-3 board)
Software	Python assembler (asm8.py) that converts program.s into program.hex; reference model (iss.py) for optional random regression
Version control	Git / GitHub

Table 1. *Tools and environment.*

2. Background

2.1 RISC and the Harvard architecture

A RISC (Reduced Instruction Set Computer) uses a small number of simple, fixed-length instructions. Only LOAD and STORE access memory; every other instruction works on registers. This regularity makes the decoder simple and makes pipelining natural, because all instructions move through the same stages. The processor uses a **Harvard** organisation: a 256 × 16-bit instruction memory and a separate 256 × 8-bit data memory, so instruction fetch (IF) and data access (MEM) never compete for the same port.

2.2 Combinational and sequential logic

Combinational logic (adders, multiplexers, decoders) produces outputs that depend only on the present inputs. Sequential logic (flip-flops and registers) stores values and changes only on the rising clock edge. A digital design is described at **Register Transfer Level (RTL)**: registers, and the combinational logic between them. In Verilog, `always @(*)` describes combinational logic and `always @(posedge clk)` with non-blocking assignments (<=) describes registers.

2.3 Pipelining

Without a pipeline, one instruction completes all of its work before the next one starts. With a pipeline, the work is split into stages separated by **pipeline registers**; each stage works on a different instruction, like an assembly line. The latency of one instruction is still five cycles, but once the pipeline is full one instruction completes every cycle. For n instructions on a k-stage pipeline the ideal time is $k + (n - 1)$ cycles instead of $n \times k$. The ideal CPI (cycles per instruction) is 1; hazards add extra cycles.

Instruction	c1	c2	c3	c4	c5	c6	c7	c8	c9
Instruction 1	IF	ID	EX	MEM	WB				
Instruction 2		IF	ID	EX	MEM	WB			
Instruction 3			IF	ID	EX	MEM	WB		
Instruction 4				IF	ID	EX	MEM	WB	
Instruction 5					IF	ID	EX	MEM	WB

Figure 1. *Ideal pipeline timing: from cycle 5 onwards one instruction completes every cycle.*

2.4 Hazards

Hazard	Cause	Solution in this design	Cost
Structural	Two stages need the same resource	Separate instruction/data memories; register file with independent read and write ports	0
Data (RAW)	An instruction needs a result that has not been written back yet	Forwarding from EX/MEM and MEM/WB; register-file write-through bypass	0
Data, load-use	A loaded value is needed by the very next instruction	1-cycle stall, then forwarding	1 cycle
Control	A taken branch makes the already fetched instructions wrong	Flush the two younger instructions	2 cycles

Table 2. *Pipeline hazards and how they are handled.*

3. Specification and Instruction Set

3.1 Processor specification

Parameter	Value
Data width	8 bits (registers, ALU, data memory)
Instruction width	16 bits, fixed
Registers	8 × 8-bit, R0–R7 (R0 is an ordinary register)
Program counter	8 bits (256 instruction addresses)
Instruction memory	256 × 16-bit ROM, initialised from program.hex with $readmemh
Data memory	256 × 8-bit RAM, read by LD and written by ST
Flags	Z (zero) and C (carry / borrow / shifted-out bit)
Pipeline	5 stages: IF, ID, EX, MEM, WB
Hazard handling	Forwarding, register-file bypass, 1-cycle load-use stall, 2-cycle branch flush
Core ports	Inputs clk, rst; outputs halted, pc_out[7:0], flags_out[1:0], instr_done

Table 3. *Specification of the processor.*

3.2 Instruction formats

Every instruction is 16 bits wide and the opcode is always in bits [15:12]. The register fields stay in the same positions in every format, so the decoder can extract them before it knows which instruction it is.

Format	Bit layout (bit 15 on the left)	Used by
R-type	`[15:12] opcode | [11:9] rd | [8:6] rs1 | [5:3] rs2 | [2:0] 000`	ADD, SUB, AND, OR, XOR
R1-type	`[15:12] opcode | [11:9] rd | [8:6] rs1 | [5:0] 000000`	SHL, SHR, LD, ST
I-type	`[15:12] opcode | [11:9] rd | [8] 0 | [7:0] imm8`	LDI, ADDI
J-type	`[15:12] opcode | [11:8] 0000 | [7:0] addr8`	JMP, BZ, BNZ

Table 4. *Instruction formats. NOP and HLT use only the opcode; all other bits are 0.*

3.3 The 16 instructions

Op	Instruction	Operation	Flags	Format
0	NOP	no operation	–	–
1	ADD rd, rs1, rs2	rd = rs1 + rs2	Z, C	R
2	SUB rd, rs1, rs2	rd = rs1 − rs2	Z, C (borrow)	R
3	AND rd, rs1, rs2	rd = rs1 AND rs2	Z (C = 0)	R
4	OR rd, rs1, rs2	rd = rs1 OR rs2	Z (C = 0)	R
5	XOR rd, rs1, rs2	rd = rs1 XOR rs2	Z (C = 0)	R
6	SHL rd, rs1	rd = rs1 << 1	Z, C = old bit 7	R1
7	SHR rd, rs1	rd = rs1 >> 1	Z, C = old bit 0	R1
8	LDI rd, imm8	rd = imm8	–	I
9	ADDI rd, imm8	rd = rd + imm8	Z, C	I
A	LD rd, [rs1]	rd = MEM[rs1]	–	R1
B	ST rd, [rs1]	MEM[rs1] = rd	–	R1
C	JMP addr8	PC = addr8	–	J
D	BZ addr8	if Z = 1 then PC = addr8	–	J
E	BNZ addr8	if Z = 0 then PC = addr8	–	J
F	HLT	stop the processor	–	–

Table 5. *Instruction set summary.*

Encoding example. `ADD R2, R2, R1`: opcode 0001, rd = 010, rs1 = 010, rs2 = 001, last bits 000 → 0001 0100 1000 1000 = `0x1488`. `LDI R1, 0x3C`: opcode 1000, rd = 001, bit 8 = 0, imm8 = 0011 1100 → `0x823C`. Both words appear in program.hex and in the simulation trace (Figure 8).

4. Architecture

4.1 Top-level organisation and file hierarchy

The core has only two inputs, `clk` and `rst`. The program is loaded into the instruction memory at the start of simulation, so the core needs no external bus. Its four outputs exist for observation: `halted`, `pc_out`, `flags_out` and `instr_done` (a pulse whenever an instruction completes write-back). The top level instantiates six sub-modules.

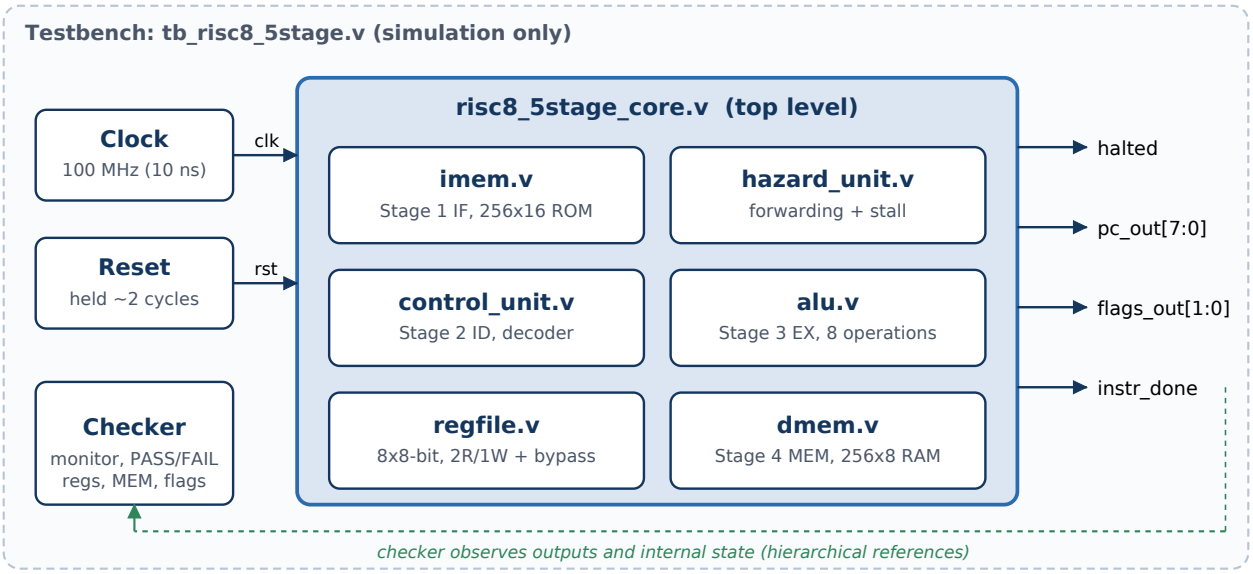


Figure 2. *System view: the core and its six sub-modules inside the testbench.*

File	Kind	Responsibility
risc8_5stage_core.v	Sequential + wiring	PC, four pipeline registers, flags, branch and halt logic; connects all modules
imem.v	Memory (ROM)	Holds the program; returns the instruction at the PC in the same cycle
control_unit.v	Combinational	Turns the 4-bit opcode into control signals
regfile.v	Sequential	8 × 8-bit registers, two read ports, one write port, write-through bypass
hazard_unit.v	Combinational	Forwarding selects and load-use stall detection
alu.v	Combinational	Arithmetic, logic and shift operations; produces Z and C
dmem.v	Memory (RAM)	256 bytes of data for LD / ST
tb_risc8_5stage.v	Testbench	Clock, reset, per-cycle monitor, automatic PASS/FAIL

Table 6. *Source files and their roles.*

4.2 The five-stage datapath

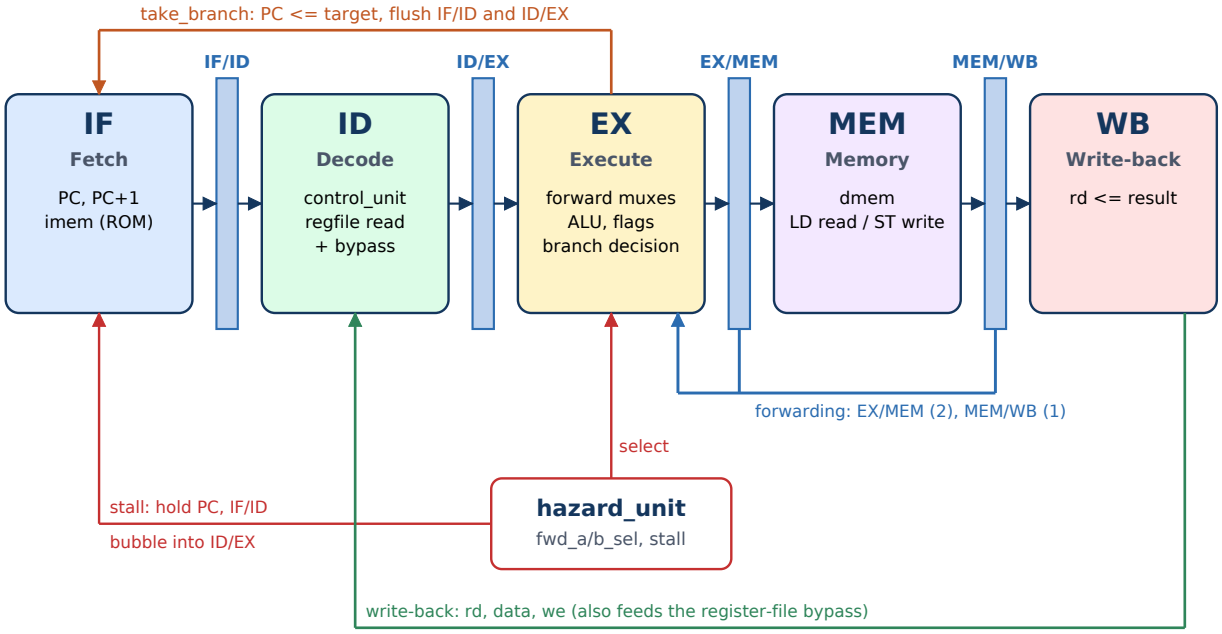


Figure 3. *Simplified five-stage datapath. Blue bars are pipeline registers; blue lines are forwarding paths, red lines are hazard-unit controls, orange is the branch redirect and green is write-back.*

- **IF:** the PC addresses the instruction ROM; at the clock edge the instruction is stored in IF/ID with `valid = 1` and the PC is updated. Next-PC priority: branch target > halt (PC frozen) > load-use stall (PC held) > PC + 1.
- **ID:** the control unit decodes the opcode and the register file is read on two ports. ADDI reads *rd* on port A (rd = rd + imm8) and ST reads *rd* on port B (the value to store).
- **EX:** two forwarding multiplexers choose each operand; a further multiplexer chooses register or immediate for ALU input B. The ALU computes the result and flags, and the branch decision is made here.
- **MEM:** LD reads the data memory, ST writes it at the clock edge; a multiplexer selects the loaded byte or the ALU result.
- **WB:** the result is written to register *rd* and is also fed back to the forwarding multiplexers and the register-file bypass.

4.3 Pipeline registers and valid bits

Each pipeline register carries a **valid bit**. A slot with `valid = 0` is a **bubble**: it moves through the pipeline like an instruction, but all of its write-enables are forced to 0, so it can never change a register, memory or the flags. Bubbles are created on purpose to stall or to cancel (flush) instructions.

Register	Contents	Width
IF/ID	instruction (16), valid (1)	17 bits
ID/EX	valid, 14 control bits, rd, two register numbers, two register values, imm8	48 bits
EX/MEM	valid, reg_we, mem_to_reg, mem_we, halt, rd, ALU result, address, store data	32 bits
MEM/WB	valid, reg_we, halt, rd, result	14 bits

Table 7. *Contents of the pipeline registers (111 flip-flops in total).*

5. RTL Implementation

The modules were written in dependency order: first the independent leaf modules (ALU, register file, memories), then the control and hazard units, and finally the top level that declares the pipeline registers and connects everything. This section highlights the important Verilog constructs of each module.

5.1 ALU (alu.v)

The ALU is purely combinational. Default values are assigned at the start of the `always @(*)` block so that every output is assigned on every path; this prevents unintended latches. A 9-bit concatenation `{c, y}` captures the carry of an addition, the borrow of a subtraction and the bit shifted out by SHL/SHR.

alu.v – core of the ALU

```
always @(*) begin
    c = 1'b0;                                       // default: no carry (avoids latches)
    case (alu_op)
        ALU_ADD  : {c, y} = {1'b0, a} + {1'b0, b}; // 9th bit = carry
        ALU_SUB  : {c, y} = {1'b0, a} - {1'b0, b}; // 9th bit = borrow
        ALU_AND  : y = a & b;
        ALU_OR   : y = a | b;
        ALU_XOR  : y = a ^ b;
        ALU_SHL  : {c, y} = {a, 1'b0};              // old a[7] -> c
        ALU_SHR  : {y, c} = {1'b0, a};              // old a[0] -> c
        ALU_PASSB: y = b;                           // LDI: copy the immediate
        default  : y = 8'h00;
    endcase
end
assign z = (y == 8'h00);                            // zero flag
```

5.2 Register file (regfile.v)

Writes are synchronous (on the rising edge, in the WB stage); reads are combinational (in the ID stage). The **write-through bypass** returns the value being written in the same cycle, which removes the distance-3 data hazard without any extra pipeline logic.

regfile.v – storage and bypass

```
reg [7:0] r [0:7];                                  // R0..R7
always @(posedge clk) begin
    if (rst)     for (i = 0; i < 8; i = i + 1) r[i] <= 8'h00;
    else if (we) r[wr_addr] <= wr_data;
end
// read with write-through bypass
assign ra_data = (we && (wr_addr == ra_addr)) ? wr_data : r[ra_addr];
assign rb_data = (we && (wr_addr == rb_addr)) ? wr_data : r[rb_addr];
```

5.3 Instruction and data memories (imem.v, dmem.v)

Both memories use asynchronous (combinational) reads, so the instruction or data is available in the same cycle as the address. The instruction ROM is filled from `program.hex` by `$readmemh`; the hex file is produced from the assembly source by `asm8.py`. The data RAM writes on the clock edge when `we` (ST) is asserted.

imem.v – program ROM

```
reg [15:0] rom [0:255];
initial begin
    for (i = 0; i < 256; i = i + 1) rom[i] = 16'h0000;   // unused words = NOP
    $readmemh(INIT_FILE, rom);                            // INIT_FILE = "program.hex"
end
assign instr = rom[addr];
```

5.4 Control unit (control_unit.v)

The control unit is a hard-wired lookup from opcode to control signals. All signals are first set to 0 (a NOP) and each case branch switches on only what its instruction needs. The signals uses_a / uses_b tell the hazard unit whether a read port is really used, so unused register fields never cause false forwarding or false stalls.

Instr.	alu_op	src_imm	ra_is_rd	rb_is_rd	uses_a	uses_b	reg_we	mem_to_reg	mem_we	flag_we	other
ADD/SUB/AND/OR/XOR	0–4				1	1	1			1	
SHL / SHR	5 / 6				1		1			1	
LDI	7	1					1				
ADDI	0	1	1		1		1			1	
LD	–				1		1	1			
ST	–			1	1	1			1		
JMP / BZ / BNZ	–										jump / br_z / br_nz
HLT	–										halt

Table 8. *Control signal table (blank = 0).*

5.5 Hazard unit (hazard_unit.v)

The hazard unit compares the source registers of the instruction in EX with the destination registers of the instructions in MEM and WB. The newest value has priority. A load in MEM is excluded from EX/MEM forwarding because its data is not ready there yet; that case is covered by the load-use stall.

hazard_unit.v – forwarding and load-use detection

```
always @(*) begin                          // operand A (operand B is identical)
    if (ex_uses_a && mem_reg_we && !mem_mem_to_reg && (mem_rd == ex_ra_addr))
        fwd_a_sel = 2'b10;                 // newest value: from EX/MEM
    else if (ex_uses_a && wb_reg_we && (wb_rd == ex_ra_addr))
        fwd_a_sel = 2'b01;                 // older value: from MEM/WB
    else
        fwd_a_sel = 2'b00;                 // no hazard: use register value
end

assign stall = id_valid && ex_valid && ex_mem_to_reg &&     // load in EX ...
               ((id_uses_a && (id_ra_addr == ex_rd)) ||     // ... and ID reads its rd
                (id_uses_b && (id_rb_addr == ex_rd)));
```

5.6 Top level (risc8_5stage_core.v)

The top level declares the four pipeline registers and updates them in a single clocked block. The branch decision is taken in EX, where the flags are already up to date. The IF/ID update follows the priority **branch flush > halt > stall > normal**, and a bubble is inserted into ID/EX in the first three cases.

risc8_5stage_core.v – PC / IF/ID priority and bubble insertion (excerpt)

```
wire take_branch = idex_valid &&
                   (idex_jump || (idex_br_z && flag_z) || (idex_br_nz && !flag_z));

if (take_branch) begin                 // 1) taken branch: redirect, flush IF/ID
    pc <= branch_target;  ifid_instr <= 16'h0000;  ifid_valid <= 1'b0;
end else if (fetch_stop) begin         // 2) HLT reached EX: stop fetching
    ifid_instr <= 16'h0000;  ifid_valid <= 1'b0;
end else if (stall) begin              // 3) load-use: hold PC and IF/ID
end else begin                         // 4) normal sequential fetch
    pc <= pc + 8'd1;  ifid_instr <= if_instr;  ifid_valid <= 1'b1;
end

if (take_branch || fetch_stop || stall) idex_valid <= 1'b0;   // bubble into ID/EX
```

6. Hazard Handling in Detail

6.1 Forwarding and the register-file bypass

Together, the two forwarding paths and the register-file bypass cover every producer–consumer distance without waiting: distance 1 is served from EX/MEM (select = 2), distance 2 from MEM/WB (select = 1) and distance 3 by the bypass inside the register file (select = 0). This is verified directly in the waveform of Figure 11.

Instruction	c1	c2	c3	c4	c5	c6	c7	c8
`LDI R1, 0x3C`	IF	ID	EX	MEM	WB			
`LDI R2, 0x0F`		IF	ID	EX	MEM	WB		
`AND R3, R1, R2`			IF	ID	EX ← fwd	MEM	WB	
`OR R4, R1, R2`				IF	ID ← byp	EX ← fwd	MEM	WB

Figure 4. *AND receives R2 from EX/MEM and R1 from MEM/WB; OR receives R1 through the register-file bypass in ID and R2 from MEM/WB.*

6.2 Load-use stall

A load produces its data only at the end of MEM. If the next instruction needs that value in EX, the data does not exist yet. The hazard unit detects *a load in EX whose destination is read by the instruction in ID* and asserts `stall` for one cycle: the PC and IF/ID keep their values and a bubble enters ID/EX. One cycle later the load is in WB and MEM/WB forwarding supplies the value.

Instruction	c1	c2	c3	c4	c5	c6	c7	c8
`LD R5, [R4]`	IF	ID	EX	MEM	WB			
`ADDI R5, 0xF0`		IF	ID	ID (wait)	EX ← fwd	MEM	WB	
`(bubble)`				bubble	bubble	bubble		
`ADDI R5, 1`			IF	IF (wait)	ID	EX ← fwd	MEM	WB

Figure 5. *Load-use stall: one bubble is inserted, then the loaded value is forwarded from MEM/WB.*

6.3 Branch flush

The core keeps fetching sequentially while a branch moves down the pipeline. The branch is resolved in EX; if it is taken, the PC is loaded with the target and the two younger instructions in IF/ID and ID/EX become bubbles. They never reach MEM or WB, so they cannot change memory or registers. A taken branch costs 2 cycles; a branch that is not taken costs nothing.

Instruction	c1	c2	c3	c4	c5	c6	c7	c8
`BZ skip1 (taken)`	IF	ID	EX	MEM	WB			
`LDI R7, 0xFF`		IF	ID	flushed				
`next sequential`			IF	flushed				
`LDI R1, 5 (target)`				IF	ID	EX	MEM	WB

Figure 6. *A taken branch flushes the two instructions fetched after it.*

6.4 Halting

When HLT reaches EX, fetching stops and younger instructions are squashed, but the older instructions in MEM and WB still complete. When HLT itself reaches WB, `halted` becomes 1 and the whole pipeline freezes. This guarantees that every instruction before HLT finishes.

7. Verification Methodology

7.1 Testbench structure

The testbench `tb_risc8_5stage.v` follows the standard structure of a self-checking testbench:

- **Timescale and signals:** `` `timescale 1ns/1ps ``; inputs of the design under test are declared as `reg`, outputs as `wire`.
- **DUT instantiation:** the core is instantiated as `dut` and the program file name is passed as the parameter `IMEM_FILE`.
- **Clock and reset:** `initial forever #5 clk = ~clk;` gives a 10 ns (100 MHz) clock; reset is released after 22 ns.
- **Per-cycle monitor:** an `always @(posedge clk)` block prints PC, the instruction in ID, the valid bits, the write-back and the flags every cycle, and counts cycles, retired instructions, stalls, flushes and forwards.
- **Automatic checking:** after `wait (halted)`, a task compares every register with its expected value using hierarchical references (`dut.u_rf.r[...]`), then checks MEM[0x20] and the flags and prints PASS/FAIL.
- **Watchdog:** a timeout ends the simulation if the core never halts.

tb_risc8_5stage.v – clock, checking task and main sequence (excerpt)

```
initial forever #5 clk = ~clk;              // 100 MHz clock

task check_reg(input [2:0] r, input [7:0] exp);
    if (dut.u_rf.r[r] !== exp) begin          // hierarchical reference into the DUT
        $display("FAIL  R%0d = %02h, expected %02h", r, dut.u_rf.r[r], exp);
        errors = errors + 1;
    end else
        $display("PASS  R%0d = %02h", r, exp);
endtask

// main sequence
#22 rst = 0;                                  // release reset after ~2 cycles
wait (halted);                                // run until HLT leaves the pipeline
check_reg(1, 8'h00);  check_reg(2, 8'h0F);  ...
if (errors == 0) $display("*** ALL TESTS PASSED ***");
$finish;
```

7.2 Test program

The directed test program (`program.s`, listed in Appendix A) is assembled to `program.hex` by `asm8.py`. It is designed to exercise every instruction class and every hazard case:

- **Part 1** – ALU operations with back-to-back dependencies (forwarding at distances 1, 2 and 3), SUB producing zero, a taken BZ and a wrong-path `LDI R7, 0xFF` that must be flushed.
- **Part 2** – a loop that computes 5 + 4 + 3 + 2 + 1 = 15 with BNZ taken four times.
- **Part 3** – ST and LD through memory address 0x20, a load-use pair, an addition that overflows (0xFF + 1) to set C and Z, a JMP over a wrong-path instruction, and HLT followed by an instruction that must never execute.

Check	Expected	Reason
R1	0x00	loop counter counted down to 0
R2	0x0F	sum 5 + 4 + 3 + 2 + 1 = 15
R3	0x01	loop constant
R4	0x20	memory address
R5	0x00	0x0F + 0xF0 = 0xFF, then 0xFF + 1 = 0x100 → 0x00
R6	0x66	SHL 0x33; wrong-path LDI R6, 0xEE and LDI R6, 0xAA never executed
R7	0x00	0x33 − 0x33; wrong-path LDI R7, 0xFF was flushed
MEM[0x20]	0x0F	stored by ST R2, [R4]
Flags Z, C	1, 1	set by ADDI R5, 1 (overflow to zero)

Table 9. *Expected final state checked automatically by the testbench.*

7.3 Vivado project setup

The seven RTL files were added as design sources and the testbench as a simulation source. `tb_risc8_5stage` was set as the simulation top and `risc8_5stage_core` as the design top. Because `$readmemh` opens `program.hex` by a relative name, the file was added to the project with file type **Memory File**, so Vivado copies it into the simulation and synthesis run directories.

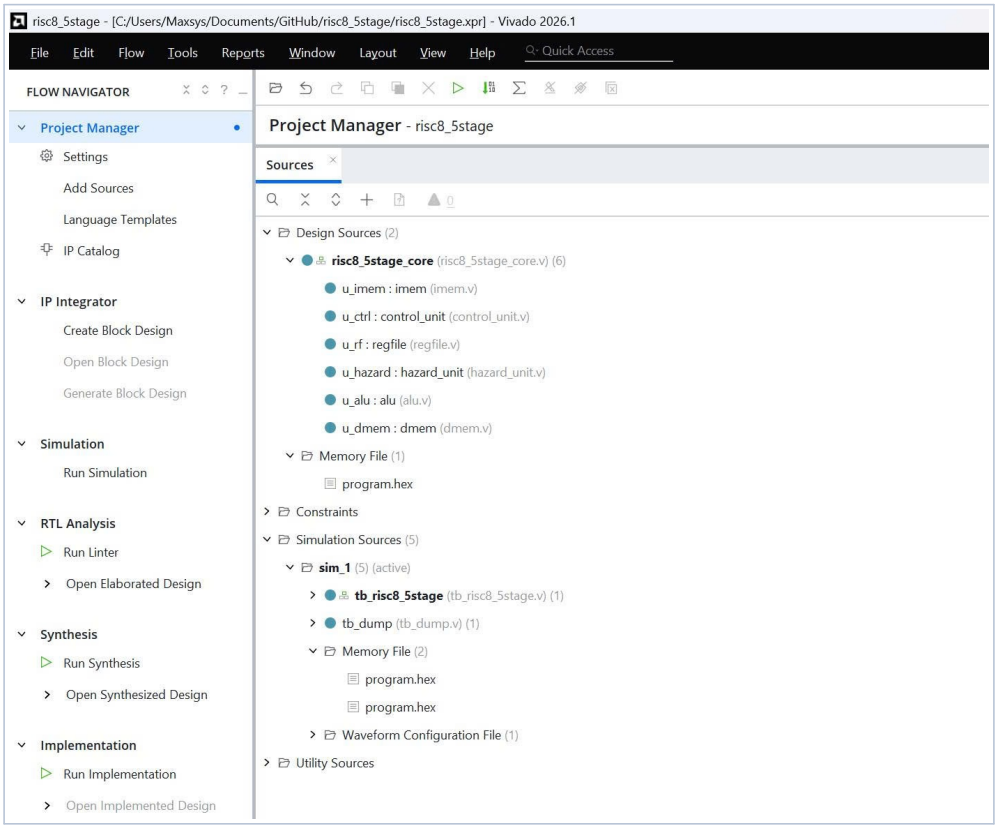


Figure 7. *Vivado Sources window: design hierarchy with the six sub-modules, the memory file and the simulation sources.*

8. Simulation Results

8.1 Cycle-by-cycle trace

Figure 8 shows the beginning of the monitor output. Each line is one clock cycle. `ID:` shows the instruction being decoded (a leading `-` marks a bubble), `EX:v` / `MEM:v` are the valid bits, `WB:w R1<=3c` means register R1 is being written with 0x3C, `fwd` marks a cycle with forwarding and `BRANCH->flush 2` a taken branch. The loop is clearly visible: R1 counts down 05, 04, 03 ... while R2 accumulates 05, 09, 0C, 0E.

```
# }
# run 1000ns
t=               25000 PC=00 | ID:-0000 | EX:v0 | MEM:v0 | WB:- R0<=00 | Z=0 C=0
t=               35000 PC=01 | ID: 823c | EX:v0 | MEM:v0 | WB:- R0<=00 | Z=0 C=0
t=               45000 PC=02 | ID: 840f | EX:v1 | MEM:v0 | WB:- R0<=00 | Z=0 C=0
t=               55000 PC=03 | ID: 3650 | EX:v1 | MEM:v1 | WB:- R0<=00 | Z=0 C=0
t=               65000 PC=04 | ID: 4850 | EX:v1 | MEM:v1 | WB:w R1<=3c | Z=0 C=0                    fwd
t=               75000 PC=05 | ID: 5ae0 | EX:v1 | MEM:v1 | WB:w R2<=0f | Z=0 C=0                    fwd
t=               85000 PC=06 | ID: 6d40 | EX:v1 | MEM:v1 | WB:w R3<=0c | Z=0 C=0                    fwd
t=               95000 PC=07 | ID: 7f80 | EX:v1 | MEM:v1 | WB:w R4<=3f | Z=0 C=0                    fwd
t=              105000 PC=08 | ID: 2fe8 | EX:v1 | MEM:v1 | WB:w R5<=33 | Z=0 C=0                    fwd
t=              115000 PC=09 | ID: d00a | EX:v1 | MEM:v1 | WB:w R6<=66 | Z=0 C=0                    fwd
t=              125000 PC=0a | ID: 8eff | EX:v1 | MEM:v1 | WB:w R7<=33 | Z=1 C=0       BRANCH->flush 2
t=              135000 PC=0a | ID:-0000 | EX:v0 | MEM:v1 | WB:w R7<=00 | Z=1 C=0
t=              145000 PC=0b | ID: 8205 | EX:v0 | MEM:v0 | WB:- R0<=3c | Z=1 C=0
t=              155000 PC=0c | ID: 8400 | EX:v1 | MEM:v0 | WB:- R0<=3c | Z=1 C=0
t=              165000 PC=0d | ID: 8601 | EX:v1 | MEM:v1 | WB:- R0<=00 | Z=1 C=0
t=              175000 PC=0e | ID: 1488 | EX:v1 | MEM:v1 | WB:w R1<=05 | Z=1 C=0
t=              185000 PC=0f | ID: 2258 | EX:v1 | MEM:v1 | WB:w R2<=00 | Z=1 C=0                    fwd
t=              195000 PC=10 | ID: e00d | EX:v1 | MEM:v1 | WB:w R3<=01 | Z=0 C=0                    fwd
t=              205000 PC=11 | ID: 8820 | EX:v1 | MEM:v1 | WB:w R2<=05 | Z=0 C=0       BRANCH->flush 2
t=              215000 PC=0d | ID:-0000 | EX:v0 | MEM:v1 | WB:w R1<=04 | Z=0 C=0
t=              225000 PC=0e | ID: 1488 | EX:v0 | MEM:v0 | WB:- R0<=05 | Z=0 C=0
t=              235000 PC=0f | ID: 2258 | EX:v1 | MEM:v0 | WB:- R0<=05 | Z=0 C=0
t=              245000 PC=10 | ID: e00d | EX:v1 | MEM:v1 | WB:- R0<=00 | Z=0 C=0
t=              255000 PC=11 | ID: 8820 | EX:v1 | MEM:v1 | WB:w R2<=09 | Z=0 C=0       BRANCH->flush 2
t=              265000 PC=0d | ID:-0000 | EX:v0 | MEM:v1 | WB:w R1<=03 | Z=0 C=0
t=              275000 PC=0e | ID: 1488 | EX:v0 | MEM:v0 | WB:- R0<=04 | Z=0 C=0
t=              285000 PC=0f | ID: 2258 | EX:v1 | MEM:v0 | WB:- R0<=04 | Z=0 C=0
t=              295000 PC=10 | ID: e00d | EX:v1 | MEM:v1 | WB:- R0<=00 | Z=0 C=0
t=              305000 PC=11 | ID: 8820 | EX:v1 | MEM:v1 | WB:w R2<=0c | Z=0 C=0       BRANCH->flush 2
t=              315000 PC=0d | ID:-0000 | EX:v0 | MEM:v1 | WB:w R1<=02 | Z=0 C=0
t=              325000 PC=0e | ID: 1488 | EX:v0 | MEM:v0 | WB:- R0<=03 | Z=0 C=0
t=              335000 PC=0f | ID: 2258 | EX:v1 | MEM:v0 | WB:- R0<=03 | Z=0 C=0
t=              345000 PC=10 | ID: e00d | EX:v1 | MEM:v1 | WB:- R0<=00 | Z=0 C=0
t=              355000 PC=11 | ID: 8820 | EX:v1 | MEM:v1 | WB:w R2<=0e | Z=0 C=0       BRANCH->flush 2
t=              365000 PC=0d | ID:-0000 | EX:v0 | MEM:v1 | WB:w R1<=01 | Z=0 C=0
t=              375000 PC=0e | ID: 1488 | EX:v0 | MEM:v0 | WB:- R0<=02 | Z=0 C=0
t=              385000 PC=0f | ID: 2258 | EX:v1 | MEM:v0 | WB:- R0<=02 | Z=0 C=0
```

Figure 8. *Tcl Console: per-cycle trace printed by the testbench monitor (first part of the run).*

8.2 Automatic check result

After the core halts, the testbench checks all registers, the data memory location and the flags. All nine checks pass, and the measured statistics are 51 cycles for 34 instructions (CPI = 1.50), one load-use stall, six taken branches and twelve forwarded operands. These figures are identical to those of the reference design.

```
------------- register / memory check -------------
PASS  R1 = 00
PASS  R2 = 0f
PASS  R3 = 01
PASS  R4 = 20
PASS  R5 = 00
PASS  R6 = 66
PASS  R7 = 00
PASS  MEM[20] = 0F
PASS  flags Z=1 C=1

cycles=51  instructions=34  CPI=1.50  load-use stalls=1  taken branches=6  forwards=12
*** ALL TESTS PASSED ***
```

Figure 9. *Final checks: every register, MEM[0x20] and the flags match the expected values – ALL TESTS PASSED.*

8.3 Complete waveform

Figure 10 shows the whole run (0–545 ns). The signals are grouped by pipeline stage: clock and reset, PC and the IF/ID instruction, the valid bits of each pipeline register, the forwarding selects, `stall`, `take_branch` and the write-back port. The `pc` trace shows the five iterations of the loop (0D–11), the six pulses of `take_branch` each followed by gaps in the valid bits, the single `stall` pulse near 430 ns, and `halted` rising at the end.

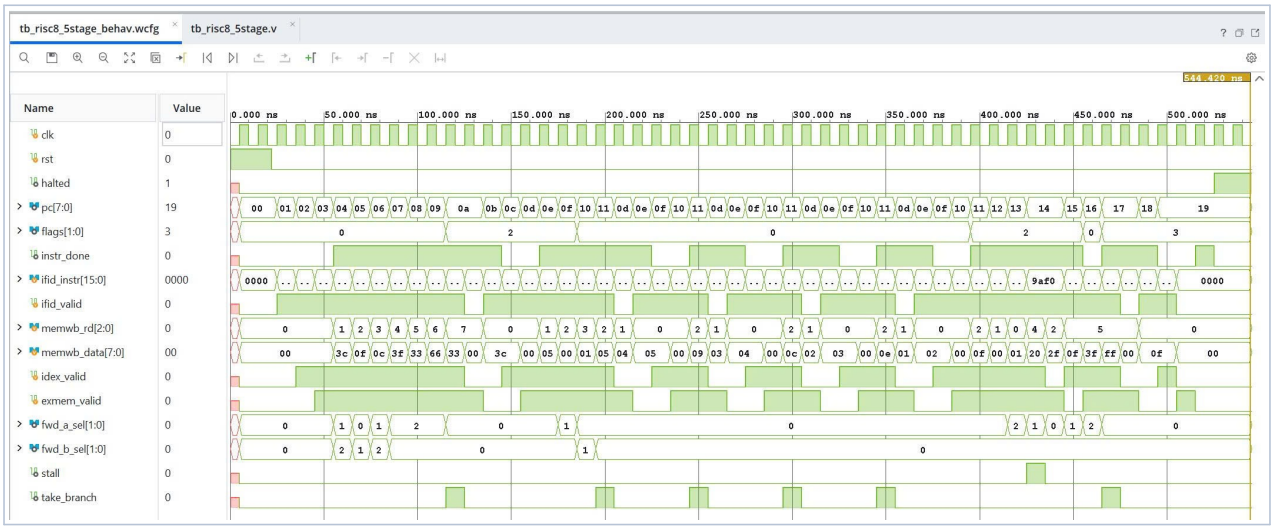


Figure 10. *Behavioral simulation waveform of the complete test program.*

8.4 Forwarding (55–135 ns)

The instruction shown in `ifid_instr` is in ID; its forwarding selects appear one cycle later, when it is in EX. The zoomed waveform confirms all three dependency distances:

EX cycle	Instruction	fwd_a_sel	fwd_b_sel	Interpretation
55–65 ns	AND R3, R1, R2 (3650)	1	2	R2 from EX/MEM (previous LDI); R1 from MEM/WB (two back)
65–75 ns	OR R4, R1, R2 (4850)	0	1	R1 three back → register-file bypass; R2 from MEM/WB
75–85 ns	XOR R5, R3, R4 (5AE0)	1	2	R4 from EX/MEM, R3 from MEM/WB
85–115 ns	SHL, SHR, SUB	2	0	each uses the result of the instruction just before it

Table 10. *Forwarding decisions observed in Figure 11.*

The write-back values `3c, 0f, 0c, 3f, 33, 66, 33` appear on consecutive cycles and `stall` stays at 0 throughout: forwarding removes all waiting for ALU-to-ALU dependencies.

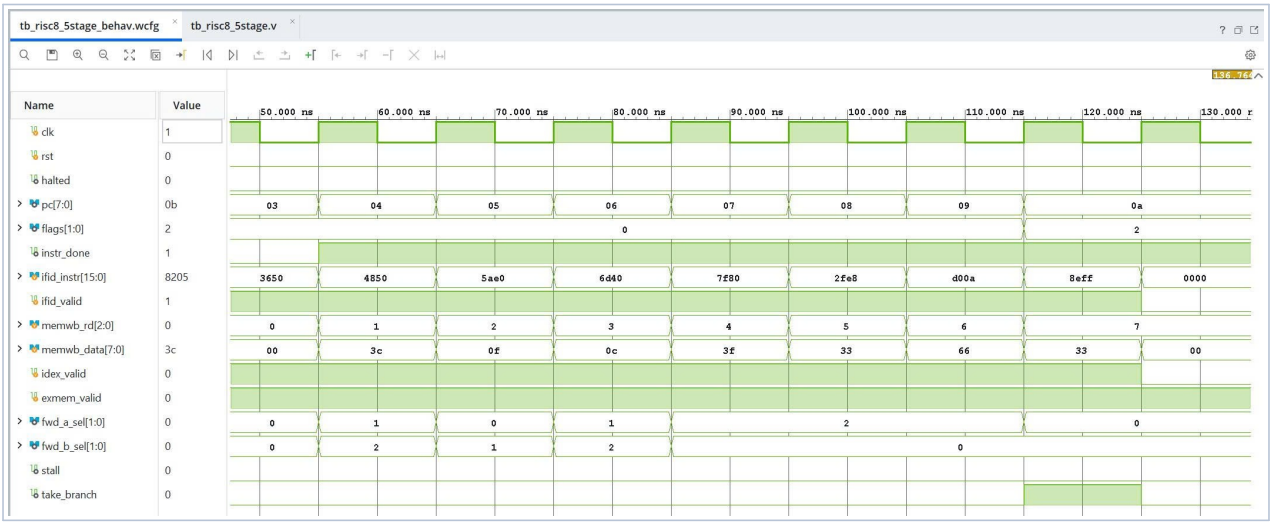


Figure 11. *Zoom on Part 1: forwarding selects 1 (MEM/WB) and 2 (EX/MEM) with no stall cycles.*

8.5 Branch flush (100–185 ns)

`SUB R7, R7, R5` (2FE8) computes 0x33 − 0x33 = 0, so the flags change to 2 (Z = 1, C = 0). `BZ skip1` (D00A) reaches EX at 120 ns and `take_branch` rises. The wrong-path `LDI R7, 0xFF` (8EFF) and the next fetched word are cancelled: `ifid_valid` drops for one cycle, and the bubbles then travel through `idex_valid` and `exmem_valid`. R7 receives only 0x33 and then 0x00 – never 0xFF – proving that the flushed instruction had no effect. Fetching resumes at the target address 0x0B with the three LDI instructions that start the loop.

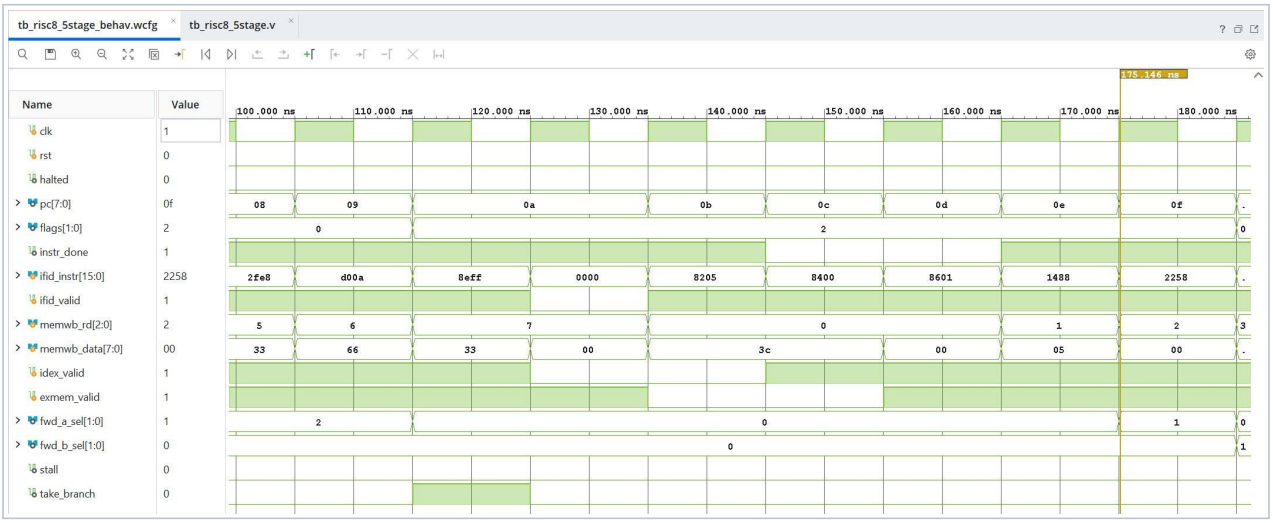


Figure 12. *Zoom on the taken BZ: two wrong-path instructions are flushed (2-cycle penalty).*

8.6 Load-use stall (405–485 ns)

`ST R2, [R4]` (B500) stores 0x0F at address 0x20 and `LD R5, [R4]` (AB00) reads it back. The next instruction, `ADDI R5, 0xF0` (9AF0), needs R5 immediately, so `stall` is asserted for exactly one clock cycle. During the stall the PC stays at 0x14 and `ifid_instr` keeps 9AF0 for two cycles, and a bubble appears in `idex_valid` and one cycle later in `exmem_valid`. When the load reaches WB (`memwb_data = 0f`), `fwd_a_sel = 1` forwards it to ADDI, which produces 0xFF. The following `ADDI R5, 1` uses `fwd_a_sel = 2` and overflows to 0x00, setting the flags to 3 (Z = 1, C = 1). Finally `JMP done` (C017) is taken and the wrong-path `LDI R6, 0xEE` (8CEE) is flushed.

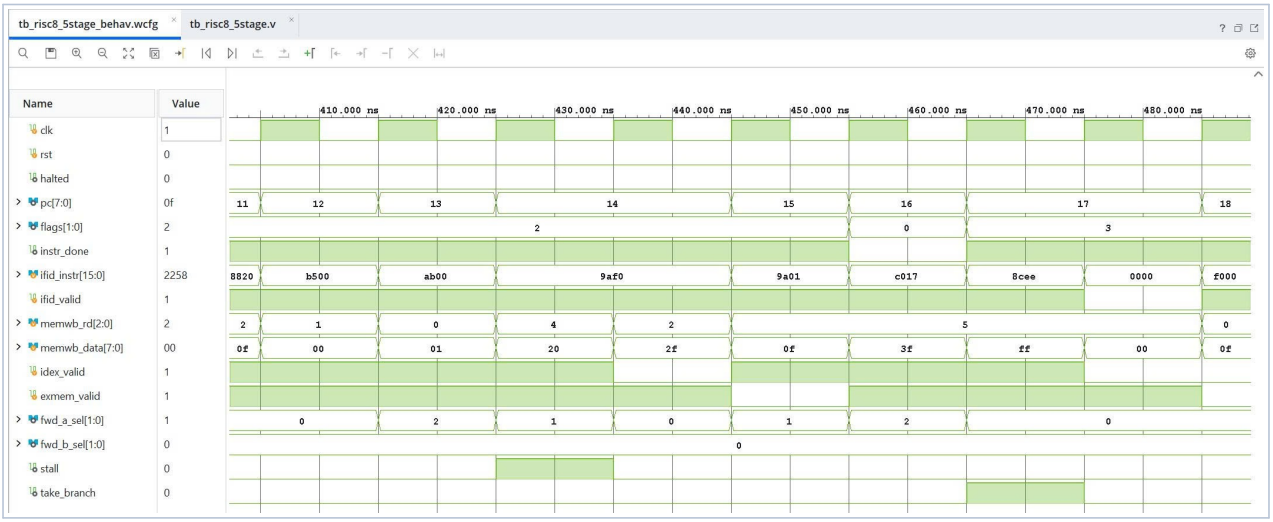


Figure 13. *Zoom on Part 3: one-cycle load-use stall followed by MEM/WB forwarding, the carry-out and the final JMP.*

8.7 Halting (465–545 ns)

HLT (F000) is decoded at about 485 ns. When it reaches EX, fetching stops: the PC freezes at 0x19 and the instruction after HLT, `LDI R6, 0xAA` (8CAA), is squashed. The older instructions drain from the pipeline, and `halted` rises when HLT itself reaches WB. The flags remain 3 (Z = 1, C = 1).

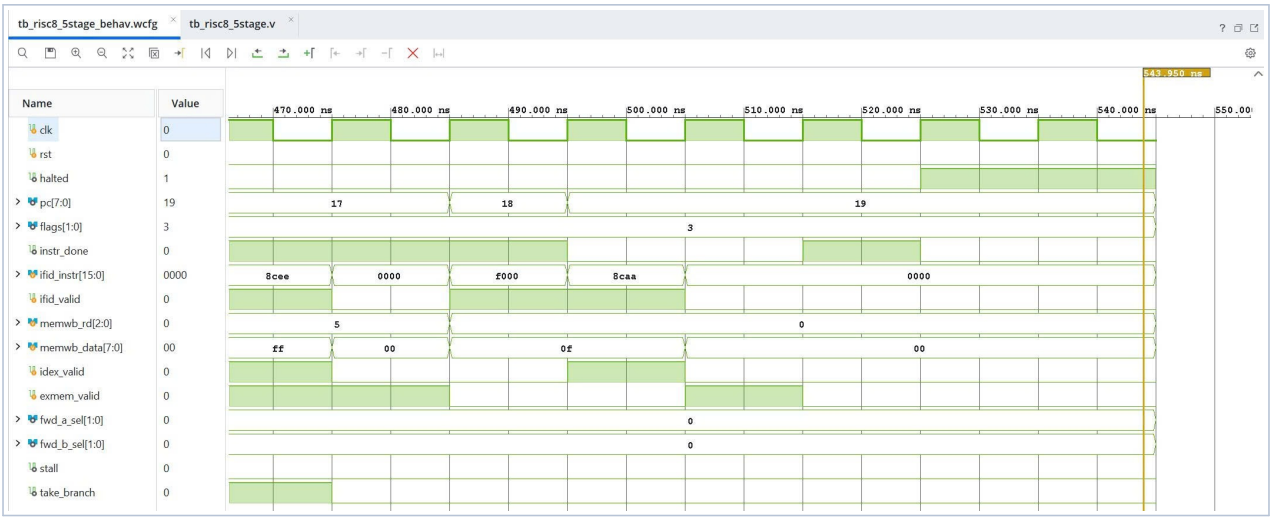


Figure 14. *End of the program: HLT stops fetching, the pipeline drains and halted is asserted.*

8.8 Final register file contents

The register array of the register file (`dut.u_rf.r[0:7]`) was added to the waveform. Its final value, read from R0 to R7, is `00, 00, 0F, 01, 20, 00, 66, 00`. This matches Table 9 exactly, and in particular R6 = 0x66 and R7 = 0x00 confirm that none of the three wrong-path LDI instructions (0xFF, 0xEE, 0xAA) was executed.

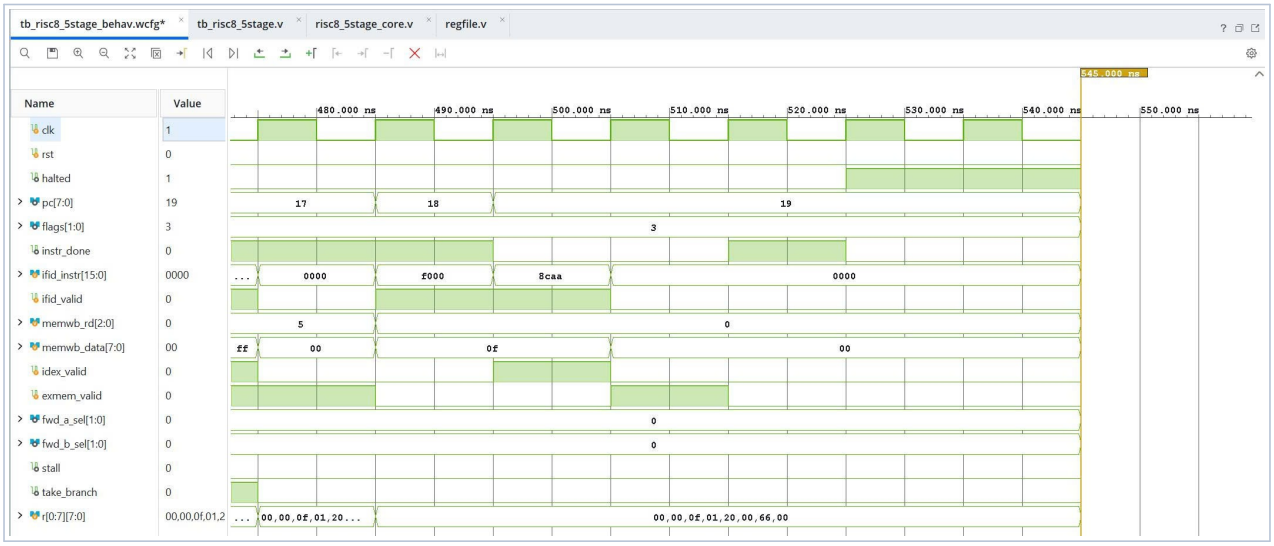


Figure 15. *Final contents of the register file R0–R7 shown in the last waveform row.*

9. Synthesis Results

9.1 Elaborated design

RTL elaboration (RTL Analysis → Open Elaborated Design → Schematic) converts the Verilog into a netlist of generic operators, multiplexers and registers before any device mapping. The schematic confirms that the code describes real hardware: it contains only registers, multiplexers, comparators and arithmetic operators, connected between the clock/reset inputs on the left and the four observation outputs on the right.

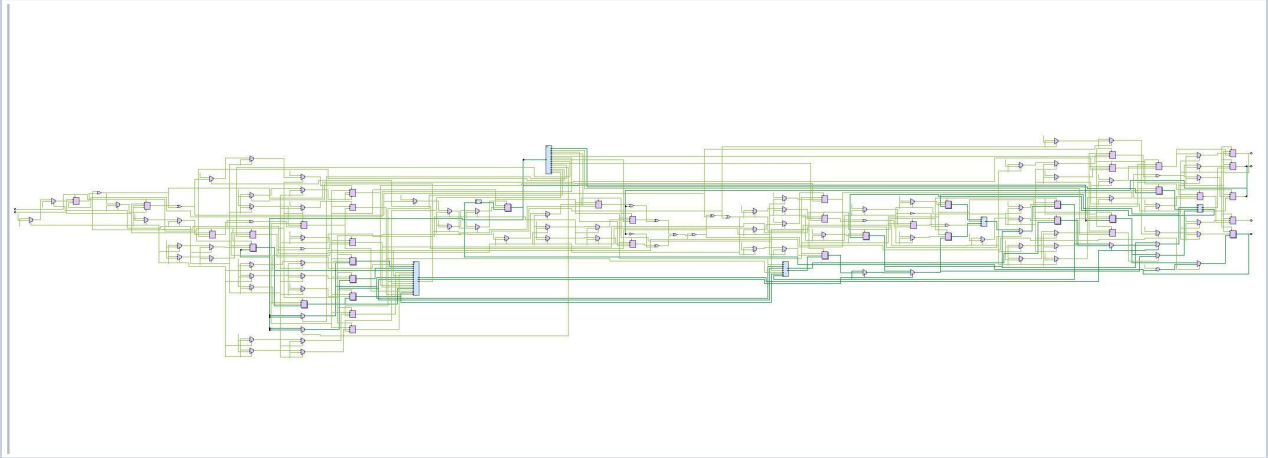


Figure 16. *Elaborated (RTL) schematic of the processor core generated by Vivado.*

9.2 Resource utilization

Synthesis completed successfully for the Artix-7 xc7a35tcpg236-1. The post-synthesis utilization is shown below.

Hierarchy

Name	Slice LUTs (20800)	Slice Registers (41600)	F7 Muxes (16300)	F8 Muxes (8150)	Bonded IOB (106)	BUFGCTRL (32)
risc8_5stage_core	238	187	24	8	14	1
u_alu (alu)	88	0	8	0	0	0
u_rf (regfile)	64	64	0	0	0	0

Figure 17. *Post-synthesis utilization by hierarchy.*

Summary

Resource	Utilization	Available	Utilization %
LUT	238	20800	1.14
FF	187	41600	0.45
IO	14	106	13.21

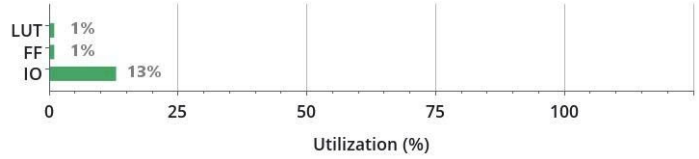


Figure 18. *Post-synthesis utilization summary.*

Resource	Used	Available	Utilization	Interpretation
Slice LUTs	238	20,800	1.14 %	ALU 88, register file 64, remainder in the top level (decoder, hazard unit, forwarding muxes, memories)
Slice registers (FF)	187	41,600	0.45 %	register file 64 (8 × 8); 123 in the top level: pipeline registers 111 + PC 8 + flags 2 + 2 control bits
F7 / F8 muxes	24 / 8	16,300 / 8,150	–	wide multiplexers used for 256-entry memory reads
Bonded IOB	14	106	13.21 %	clk, rst, halted, pc_out[7:0], flags_out[1:0], instr_done
BUFGCTRL	1	32	3 %	global clock buffer

Table 11. *Post-synthesis resource utilization (xc7a35tcpg236-1).*

Several observations follow from these numbers:

- The **ALU uses 0 flip-flops**, confirming that it is purely combinational, and the **register file uses exactly 64 flip-flops** (8 registers × 8 bits).
- The 123 flip-flops of the top level match the sum of the pipeline-register widths in Table 7 (17 + 48 + 32 + 14 = 111) plus the PC, the two flags and two control bits (draining, halted).
- The 256 × 8 data memory was mapped to **distributed (LUT) RAM** (eight RAM256X1S primitives) instead of 2,048 flip-flops, because its read is combinational and its write is synchronous.
- The small instruction and control modules do not appear as separate rows: Vivado merged them into the top level during optimisation. The program is fixed, so the instruction ROM becomes constant logic.
- The whole processor occupies about 1 % of the logic of this small FPGA.

10. Performance Analysis

The dynamic instruction count of the test program is 34 (9 in Part 1, 3 + 5 × 3 = 18 in Part 2 and 7 in Part 3). The 51 measured cycles can be explained exactly:

Component	Cycles	Explanation
Retired instructions	34	ideal: one instruction completes per cycle
Pipeline fill	4	the first instruction needs five cycles to reach WB
Taken-branch flushes	12	6 taken branches (1 BZ, 4 BNZ, 1 JMP) × 2 cycles
Load-use stall	1	LD R5 followed immediately by ADDI R5
Total	**51**	**CPI = 51 / 34 = 1.50**

Table 12. *Breakdown of the measured cycle count.*

Without forwarding, each of the 12 forwarded operands would have required one or two stall cycles, so forwarding is the main reason the CPI stays close to the ideal value. The largest remaining cost is control hazards (12 of 17 extra cycles). Resolving branches in ID or adding a simple branch predictor would reduce this penalty at the cost of extra hardware. With the 10 ns testbench clock the program runs in 51 × 10 ns = 510 ns of simulated time.

11. Challenges and Lessons Learned

Issue observed	Cause	Resolution
The core executed only NOPs; PC ran past the program and halted stayed 0	$readmemh could not find program.hex from the XSim run directory	Added program.hex to the project and set its file type to Memory File so Vivado exports it
The first simulation showed unknown (X) values and no PASS/FAIL output	tb_dump (the random-test testbench) had been selected as simulation top	Set tb_risc8_5stage as top in Simulation Sources
“Process cannot access simulate.log” when relaunching	The previous simulation was still open and held the log file	Closed it with close_sim before starting a new run
Synthesis needed the program as well	The memory file was initially only in the simulation file set	Added program.hex to the design sources as a Memory File

Table 13. *Practical issues met in Vivado and how they were solved.*

Key lessons from the project:

- Assigning default values at the start of combinational `always @(*)` blocks is a simple and reliable way to avoid unintended latches.
- Carrying a **valid bit** with every instruction makes stalls and flushes simple: an instruction is cancelled just by clearing its valid bit, and all of its write-enables are gated.
- A self-checking testbench with a per-cycle text trace is far faster to debug than reading waveforms alone; the waveforms then confirm the exact timing of each hazard.
- Hierarchical references (`dut.u_rf.r[1]`) let a testbench check internal state without adding extra ports to the design.
- Waveforms are easier to read when signals are grouped by pipeline stage and shown in hexadecimal.

12. Conclusion

An 8-bit RISC processor with a five-stage pipeline was implemented in synthesizable Verilog and verified in Vivado. The directed self-checking testbench passed all checks on registers, memory and flags. The waveforms demonstrate each hazard mechanism: forwarding at distances 1 and 2, the register-file bypass at distance 3, a one-cycle load-use stall, two-cycle branch flushes in which wrong-path instructions have no effect, and an orderly halt. The measured CPI of 1.50 is fully explained by pipeline fill, six taken branches and one stall. Synthesis for Artix-7 used 238 LUTs and 187 flip-flops, about 1 % of the device.

References

- GUVI × HCLTech, *8-bit RISC Processor Core with a 5-Stage Pipeline – Architecture, Block Diagrams and Implementation Guide* (project reference material).
- D. A. Patterson and J. L. Hennessy, *Computer Organization and Design*, Morgan Kaufmann (chapter on pipelining and hazards).
- AMD, *Vivado Design Suite User Guide: Logic Simulation* (UG900) and *Synthesis* (UG901).

Appendix A. Test Program Listing

Addr	Hex	Assembly	Comment
00	823C	LDI R1, 0x3C	R1 = 0x3C
01	840F	LDI R2, 0x0F	R2 = 0x0F
02	3650	AND R3, R1, R2	R3 = 0x0C (forwarding)
03	4850	OR R4, R1, R2	R4 = 0x3F
04	5AE0	XOR R5, R3, R4	R5 = 0x33
05	6D40	SHL R6, R5	R6 = 0x66
06	7F80	SHR R7, R6	R7 = 0x33
07	2FE8	SUB R7, R7, R5	R7 = 0x00, Z = 1
08	D00A	BZ skip1	taken
09	8EFF	LDI R7, 0xFF	wrong path, flushed
0A	8205	LDI R1, 5	skip1: loop counter
0B	8400	LDI R2, 0	sum
0C	8601	LDI R3, 1	constant 1
0D	1488	ADD R2, R2, R1	loop: sum += counter
0E	2258	SUB R1, R1, R3	counter − 1, sets Z
0F	E00D	BNZ loop	taken 4 times
10	8820	LDI R4, 0x20	address
11	B500	ST R2, [R4]	MEM[0x20] = 0x0F
12	AB00	LD R5, [R4]	R5 = 0x0F
13	9AF0	ADDI R5, 0xF0	load-use: R5 = 0xFF
14	9A01	ADDI R5, 1	R5 = 0x00, Z = 1, C = 1
15	C017	JMP done	taken
16	8CEE	LDI R6, 0xEE	wrong path, flushed
17	F000	HLT	done: stop
18	8CAA	LDI R6, 0xAA	never executed

Table 14. *Test program (sw/program.s) and its machine code (program.hex).*

Appendix B. Repository Structure

Project folder layout

```
risc8_5stage/
|-- rtl/   alu.v  regfile.v  imem.v  dmem.v  control_unit.v  hazard_unit.v  risc8_5stage_core.v
|-- tb/    tb_risc8_5stage.v  (directed self-checking test)   tb_dump.v  (random regression)
|-- sw/    program.s  asm8.py (assembler)  iss.py (reference model)  random_test.py
|-- docs/  screenshots used in this report
|-- program.hex          machine code loaded by imem.v
|-- files.f, run_sim.sh, run_sim.bat, sim_log.txt, README.md
```